Project 2: Market Basket Analysis using Apriori and PCY

Zhuoxin Ge 83511A

March 3, 2026

1 Introduction

Frequent itemset mining is a fundamental problem in data mining. This project implements and compares the Apriori and PCY algorithms for discovering frequent item pairs in a transactional dataset derived from the IMDB Top 1000 Movies dataset.

Each movie is considered a basket, and the actors listed in `Star1`, `Star2`, `Star3`, and `Star4` are treated as items.

The objectives of this project are:

- To implement Apriori and PCY correctly.
- To experimentally compare their efficiency.
- To analyze scalability with respect to data size.

2 Dataset Description

The dataset used is the IMDB Top 1000 Movies dataset obtained from Kaggle.

Only the following attributes were considered:

- Star1
- Star2
- Star3
- Star4

After preprocessing:

- Number of baskets: 1000
- Items per basket: 4 actors

Each basket represents one movie and contains four actor names.

3 Data Organization and Preprocessing

The dataset was transformed into transactional form as follows:

- Rows with missing actor values were removed.
- Leading and trailing whitespace was stripped from actor names.
- Each movie was converted into a set of actors.

This ensures compatibility with frequent itemset mining algorithms.

4 Algorithms

4.1 Apriori

For frequent pair mining, Apriori proceeds in two passes:

1. Count frequent 1-itemsets.
2. Generate candidate 2-itemsets from frequent items.
3. Count support and prune infrequent pairs.

Candidate generation follows:

$$C_2 = \{\{i, j\} \mid i, j \in L_1\}$$

Apriori may suffer from combinatorial explosion when minimum support decreases, since the number of frequent 1-itemsets increases and candidate pairs grow quadratically.

4.2 PCY

PCY improves Apriori by introducing hashing:

- During the first pass, candidate pairs are hashed into buckets.
- Buckets are counted and stored.
- A bitmap marks frequent buckets.
- In the second pass, only pairs mapping to frequent buckets are counted.

This reduces the number of candidate pairs that need explicit counting.

5 Experimental Setup

The experiments were conducted in Python on Google Colab.

All experiments were executed using the same random seed and identical hardware configuration. The full implementation is provided in the accompanying Jupyter notebook.

Two experimental dimensions were evaluated:

1. Effect of minimum support.
2. Scalability with increasing dataset size.

Unless otherwise stated, the number of buckets was set to 5000.

6 Experiment 1: Effect of Minimum Support

Minimum support was varied from 3 to 10.

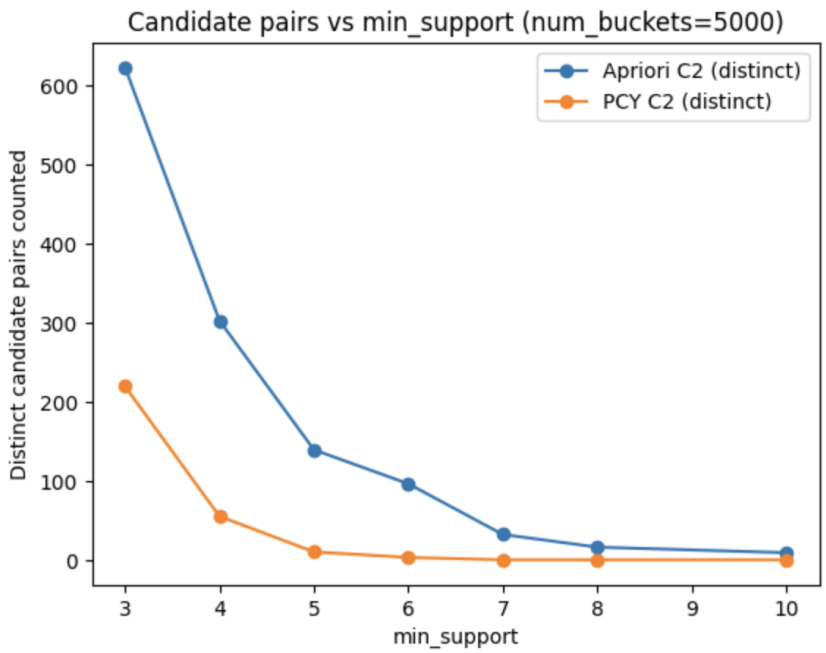

Figure 1: Distinct candidate pairs vs minimum support (5000 buckets)

At support = 3:

- Apriori generated approximately 622 distinct candidate pairs.
- PCY reduced this to approximately 229 pairs.

This represents a reduction of more than 60% in candidate space.

As minimum support increases, both algorithms generate fewer candidates.This is because higher support thresholds reduce the number of frequent 1-itemsets, thereby limiting the number of possible 2-item combinations.

However, PCY consistently produces significantly fewer candidates due to bucket-based pruning.

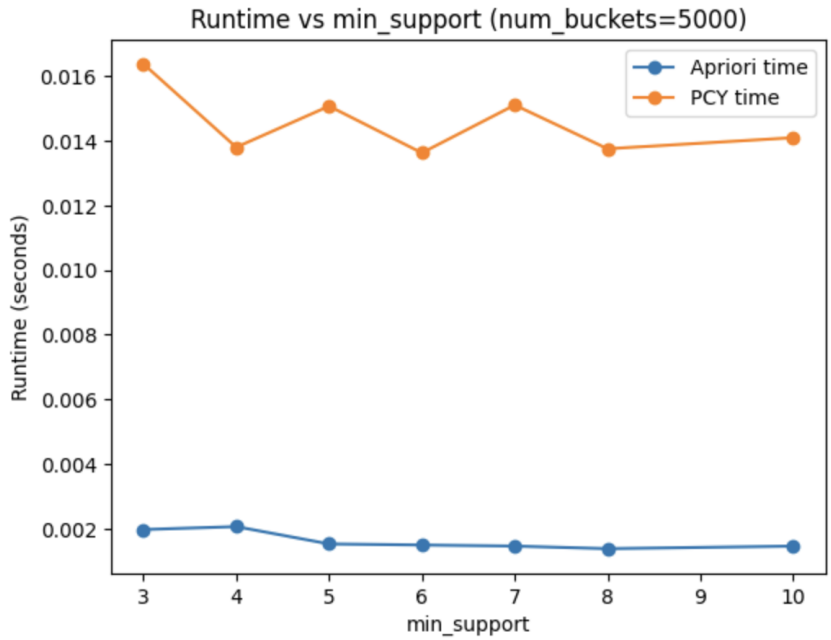

Figure 2: Runtime vs minimum support

Although PCY significantly reduces the number of candidate pairs, its runtime remains higher than Apriori in this small dataset due to hashing overhead introduced in the first pass.Since each basket contains only four items and the dataset size is limited, the computational savings from pruning are not sufficient to outweigh the additional hashing cost.

7 Experiment 2: Scalability (Fixed Support and Buckets)

Dataset size was increased by duplicating baskets (2x, 5x, 10x), while keeping minimum support and number of buckets fixed.

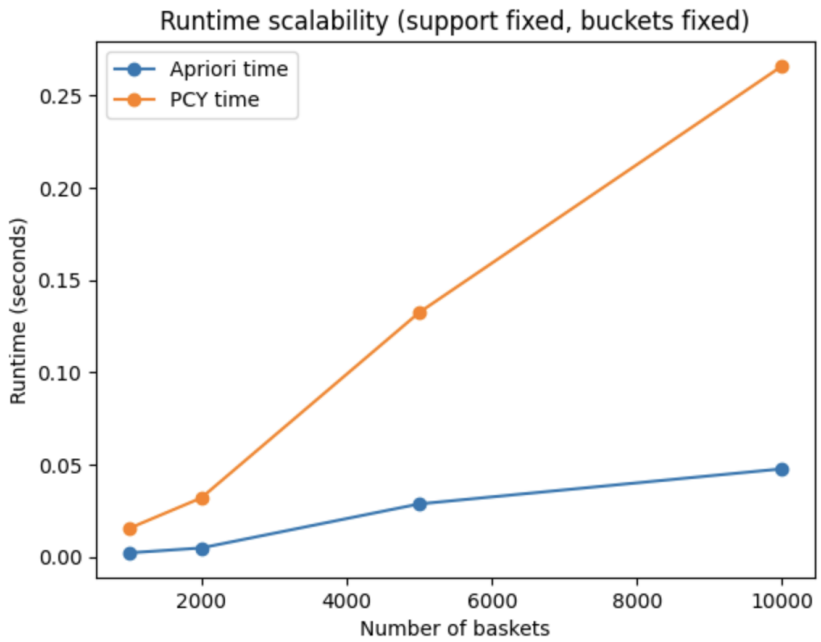

Figure 3: Runtime scalability (support fixed, buckets fixed)

Runtime increases approximately linearly with the number of baskets for both algorithms, indicating linear scalability with respect to data size.

Since minimum support remains fixed and baskets are duplicated proportionally, the set of frequent 1-itemsets and the number of candidate pairs remain unchanged.

However, because the number of buckets is fixed, the hashing space does not expand with the dataset. As the dataset grows, bucket counts increase proportionally, but collision effects are not reduced. Consequently, the pruning effectiveness of PCY does not improve, while the hashing overhead grows linearly with data size.

This explains why PCY exhibits higher runtime compared to Apriori under this configuration.

8 Experiment 3: Scalability with Scaled Support and Buckets

To simulate realistic scaling, minimum support and bucket count were scaled proportionally with dataset size.

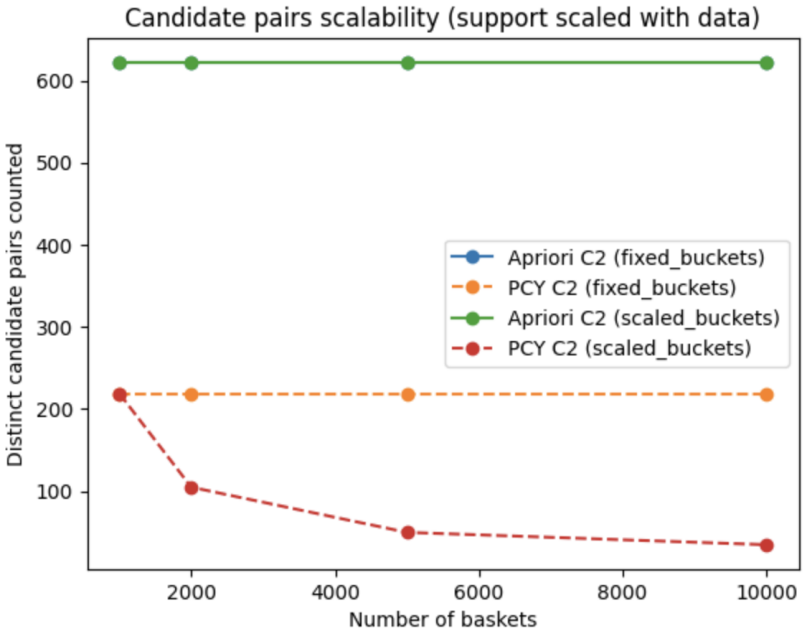

Figure 4: Candidate pairs scalability (support scaled with data)

Under this configuration:

- Apriori candidate count remains approximately constant.
- PCY candidate count decreases significantly as bucket count increases.

Increasing the number of buckets reduces hash collisions and improves pruning efficiency. For example, with 10,000 baskets and 50,000 buckets, PCY reduces candidate pairs dramatically compared to the fixed-bucket setting.

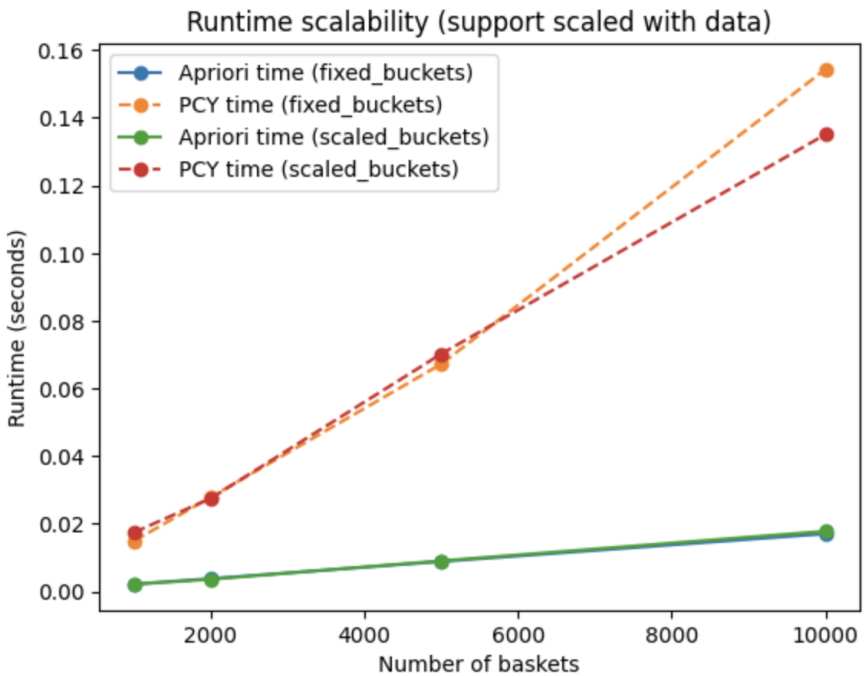

Figure 5: Runtime scalability (support scaled with data)

Runtime grows approximately linearly with dataset size for both algorithms, confirming linear scalability.

When bucket size scales proportionally with data, PCY benefits from reduced hash collisions, which improves pruning efficiency. Although PCY still incurs additional hashing overhead compared to Apriori, its runtime under scaled buckets is lower than in the fixed-bucket setting, demonstrating improved scalability.

9 Discussion

The experiments align with theoretical expectations of frequent itemset mining.

Lower minimum support increases the number of frequent 1-itemsets, which leads to quadratic growth ($O(|L_1|^2)$) in candidate 2-itemsets for Apriori.This explains the rapid expansion of the candidate space when support decreases.

PCY mitigates this combinatorial growth by hashing pairs into buckets and pruning infrequent candidates early using a bitmap structure.

The effectiveness of PCY depends strongly on bucket configuration:

• With fixed bucket size, increasing dataset size leads to more hash collisions, reducing pruning effectiveness.

• When bucket size scales with dataset size, hash collisions decrease and pruning efficiency improves significantly.

For relatively small datasets, runtime improvements remain limited because hashing introduces additional computational overhead. However, under larger-scale configurations with sufficient bucket space, PCY demonstrates stronger scalability advantages by significantly reducing candidate space while maintaining linear runtime growth.

Both algorithms consistently produce identical frequent itemsets, confirming correctness of implementation.

10 Conclusion

This project implemented and experimentally compared Apriori and PCY for frequent pair mining.

Results demonstrate that:

• Apriori experiences rapid candidate growth as minimum support decreases due to quadratic candidate generation.

• PCY significantly reduces candidate space through hashing-based pruning.

• Scalability improves substantially when bucket size scales proportionally with dataset size, reducing hash collisions.

• Both algorithms consistently return identical frequent itemsets, validating correctness.

Overall, while PCY may incur additional hashing overhead for small datasets, it becomes increasingly advantageous as dataset size grows and bucket configuration is properly chosen. These findings confirm the theoretical design principles underlying PCY and highlight the importance of hash space configuration in scalable frequent itemset mining.

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.